

Data Structures and Analysis of Algorithms

CST 225-3

Assignment 01

(Group No: CST 10)

Computer Science and Technology

Department of Computer Science and Informatics

Faculty of Applied Sciences

Uva Wellassa University of Sri Lanka

2026

1. Introduction and Problem Overview

A city planning authority wants to connect six critical locations: Hospital (A), School (B), Mall (C), Police Station (D), Railway Station (E), and Airport (F). Each candidate road has a construction or travel cost. The first objective is to connect every location at minimum total cost using a Minimum Spanning Tree (MST). The second objective is to determine the fastest emergency routes from Hospital A using single-source shortest paths.

- Prim's and Kruskal's algorithms are used to design the minimum-cost connected road network.
- Dijkstra's algorithm is used to find the shortest emergency route from Hospital A to every location.
- A flood scenario tests resilience by closing road D-E and identifying alternative routes.

2. Original Weighted Graph

Original Weighted Road Graph

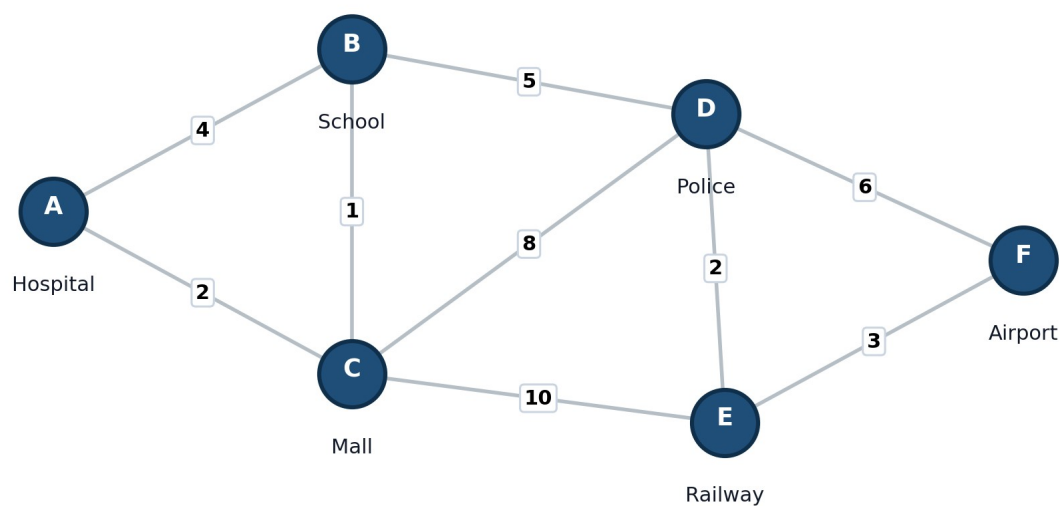


Figure 1. Original weighted undirected road graph

Road	Cost	Road	Cost	Road	Cost
A-B	4	A-C	2	B-C	1
B-D	5	C-D	8	C-E	10
D-E	2	D-F	6	E-F	3

3. Minimum Spanning Tree Using Prim's Algorithm

Prim's algorithm begins at Hospital A and grows one tree. At each step, it selects the cheapest road connecting a visited location to an unvisited location. The process stops after selecting $V-1 = 5$ edges for the six vertices.

Prim's Algorithm MST (start at A)

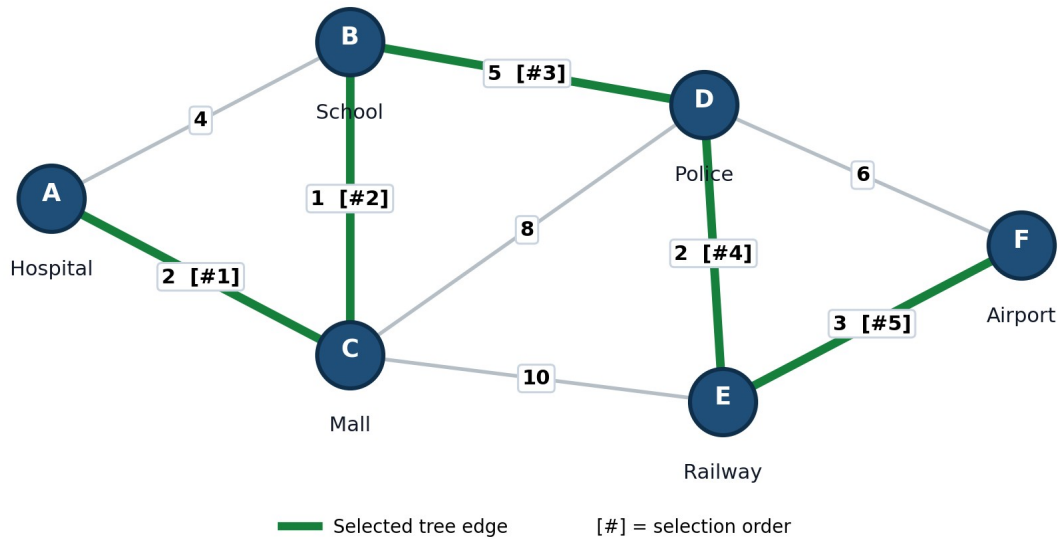


Figure 2. Prim MST; [#] shows the edge-selection order

Step	Visited set	Selected edge	Cost	Total
1	A, C	A-C	2	2
2	A, B, C	C-B	1	3
3	A, B, C, D	B-D	5	8
4	A, B, C, D, E	D-E	2	10
5	A, B, C, D, E, F	E-F	3	13

Prim MST result: {A-C(2), C-B(1), B-D(5), D-E(2), E-F(3)}; total cost = 13.

4. MST Using Kruskal's Algorithm

Kruskal's algorithm sorts all roads by increasing cost. It adds the next cheapest road only if that road does not create a cycle. Union-Find can be used to test whether two vertices already belong to the same component.

Kruskal's Algorithm MST

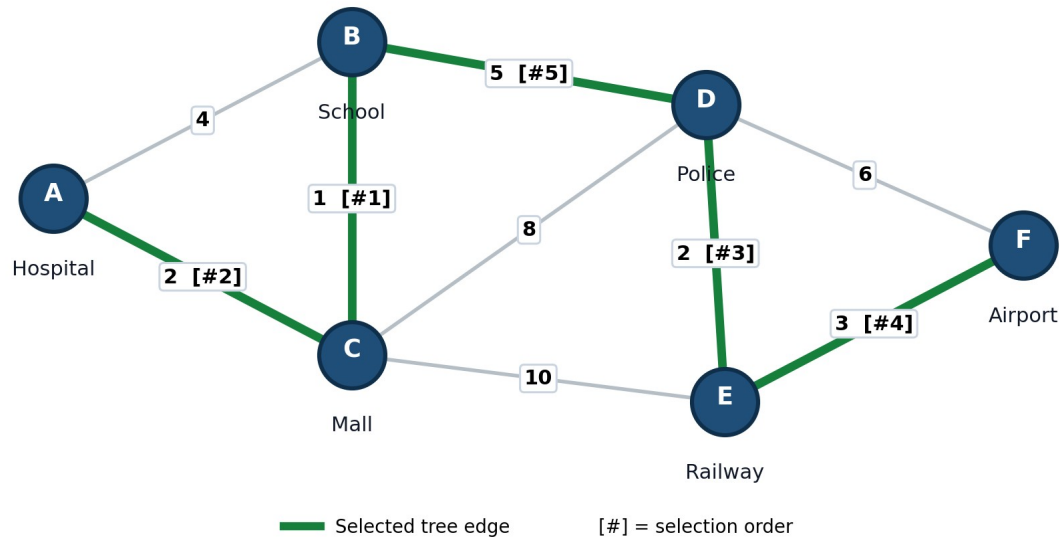


Figure 3. Kruskal MST; [#] shows the accepted-edge order

Order	Road	Cost	Decision	Reason
1	B-C	1	Accept	Connects B and C
2	A-C	2	Accept	Adds A
3	D-E	2	Accept	Connects D and E
4	E-F	3	Accept	Adds F
5	A-B	4	Reject	Creates cycle A-B-C-A
6	B-D	5	Accept	Joins both components

Kruskal MST result: {B-C(1), A-C(2), D-E(2), E-F(3), B-D(5)}; total cost = 13.

5. Comparison of Prim's and Kruskal's Algorithms

Both algorithms produce the same minimum spanning tree and total cost for this graph, but their selection strategies and supporting data structures are different.

Aspect	Prim's Algorithm	Kruskal's Algorithm
Approach	Grows one connected tree from a start vertex.	Builds a forest and merges components.
Selection	Cheapest edge from visited to unvisited set.	Globally cheapest cycle-free edge.
Cycle control	Visited set prevents adding an old vertex.	Union-Find detects cycles.
Data structures	Adjacency list/matrix and min-priority queue.	Sorted edge list and disjoint set.
Complexity	$O(E \log V)$ with adjacency list and heap; $O(V^2)$ with matrix.	$O(E \log E)$, mainly for edge sorting.
Suitability	Often preferred for dense connected graphs.	Often preferred for sparse edge-list graphs.
Starting point	Requires a chosen start vertex.	Does not require a start vertex.
This problem	Natural when construction begins at Hospital A.	Simple because only nine roads are sorted.

5.1 Final comparison result

- Prim's selection order: A-C, C-B, B-D, D-E, E-F.
- Kruskal's accepted order: B-C, A-C, D-E, E-F, B-D.
- Both MSTs contain the same five roads and have total cost 13.

6. Shortest Paths Using Dijkstra's Algorithm

Dijkstra's algorithm starts with $d(A)=0$ and all other distances set to infinity. It repeatedly finalizes the unvisited vertex with the smallest tentative distance and relaxes all adjacent roads. All road costs are non-negative, so Dijkstra's algorithm is valid.

Dijkstra's Shortest-Path Tree from Hospital A

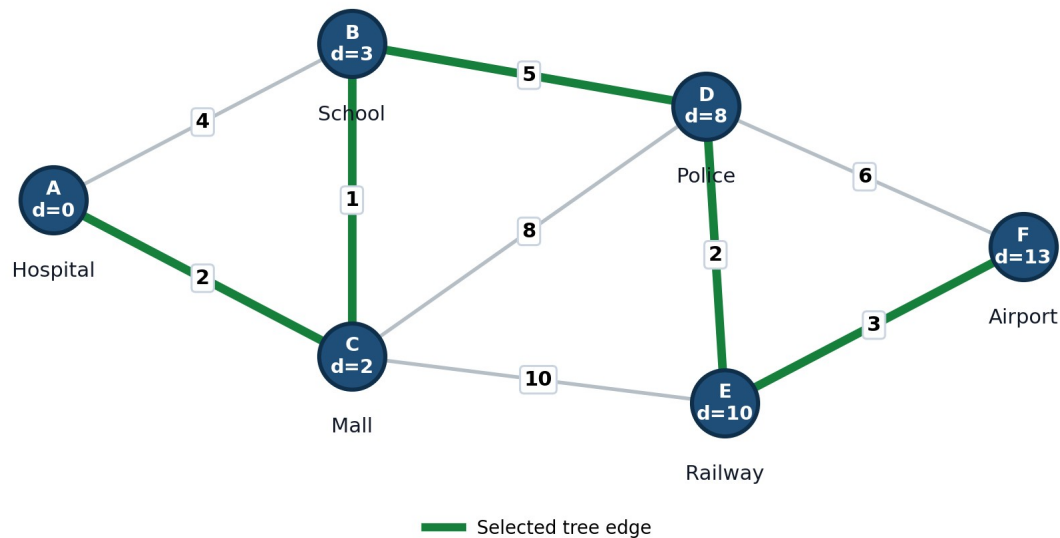


Figure 4. Dijkstra shortest-path tree; each node shows its final distance from A

Location	Shortest path from A	Calculation	Distance
A Hospital	A	Start	0
B School	A-C-B	$2 + 1$	3
C Mall	A-C	2	2
D Police	A-C-B-D	$2 + 1 + 5$	8
E Railway	A-C-B-D-E	$2 + 1 + 5 + 2$	10
F Airport	A-C-B-D-E-F	$2 + 1 + 5 + 2 + 3$	13

Dijkstra result: Distances from A: A=0, B=3, C=2, D=8, E=10, F=13.

7. Dijkstra's Algorithm - Iteration Table

The table below shows the shortest tentative distances after each vertex is selected. A dash represents infinity or a value that has not yet been discovered.

Selected	d(A)	d(B)	d(C)	d(D)	d(E)	d(F)
Initial	0	-	-	-	-	-
A	0	4	2	-	-	-
C	0	3	2	10	12	-
B	0	3	2	8	12	-
D	0	3	2	8	10	14
E	0	3	2	8	10	13
F	0	3	2	8	10	13

7.1 Important relaxations

- Through C, the distance to B improves from 4 to $2+1=3$.
- Through B, the distance to D improves from 10 to $3+5=8$.
- Through D, the distance to E improves from 12 to $8+2=10$.
- Through E, the distance to F improves from 14 to $10+3=13$.

8. Flood Simulation: Road D-E Closed

The flood removes D-E(2), a low-cost road used by both the original MST and the original shortest routes to E and F. The city remains connected because C-E, D-F, and E-F are still available.

Flood Scenario: D-E Closed and Alternative Corridors

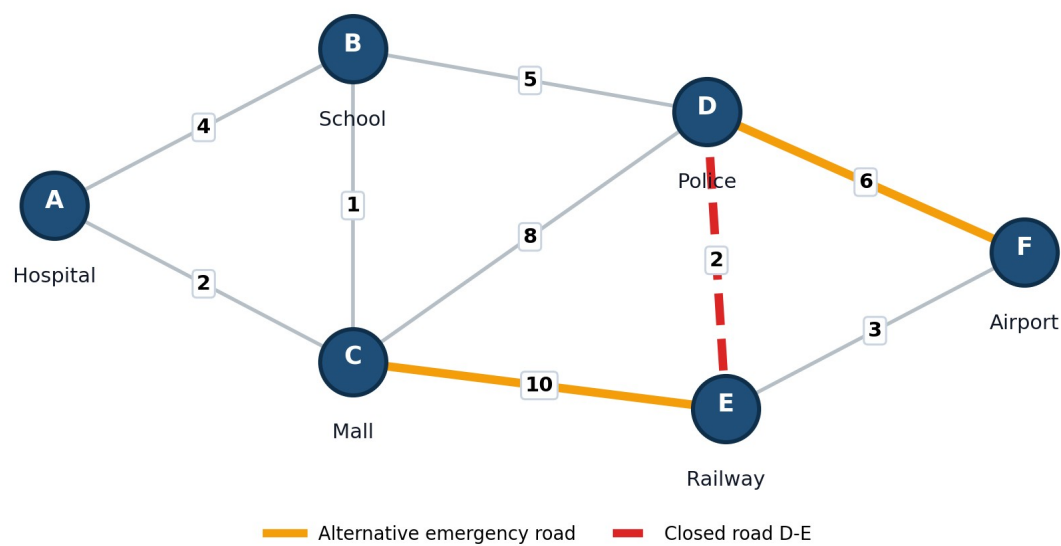


Figure 5. Flood graph: D-E is closed; C-E and D-F become key alternatives

Destination	Before	After	Impact	New route from A
B	3	3	No change	A-C-B
C	2	2	No change	A-C
D	8	8	No change	A-C-B-D
E	10	12	+2	A-C-E
F	13	14	+1	A-C-B-D-F

Flood impact: Railway distance rises to 12; Airport distance rises to 14.

9. Alternative Network and Recommendations

- Use A-C-E for Hospital-to-Railway emergencies at cost 12.
- Use A-C-B-D-F for Hospital-to-Airport emergencies at cost 14.
- The flood-safe MST is {B-C(1), A-C(2), E-F(3), B-D(5), D-F(6)} with total cost 17.
- The network cost increases by 4 units, from 13 to 17, showing that D-E is a critical road.
- Flood-proof or elevate D-E and maintain C-E and D-F as emergency backup corridors.

10. Java Implementation

The accompanying CityRoadNetwork.java program implements the weighted graph with an adjacency list. Prim's and Dijkstra's algorithms use priority queues. Kruskal's algorithm uses edge sorting and Union-Find. The closeRoad method removes D-E and reruns the shortest-path and MST calculations.

Program verification	Output
Prim MST cost	13
Kruskal MST cost	13
Dijkstra distances A-F	0, 3, 2, 8, 10, 13
Flood distances A-F	0, 3, 2, 8, 12, 14
Flood-safe MST cost	17

11. Conclusion

Prim's and Kruskal's algorithms independently produce the same minimum road network with total cost 13. Dijkstra's algorithm provides efficient routes from Hospital A. Closing D-E does not disconnect the city, but it increases both travel distance and infrastructure cost. Therefore, C-E and D-F should be maintained as emergency alternatives while D-E is strengthened against floods.